ASSIGNMENT 1 · GROUP REPORT

# Structured Bug Report Helper

## Project definition, method selection and collaborative workflow trial

AUT · Contemporary Methods in Software Engineering

Revised 9 November 2026 · Topic approval confirmed by the owner

| Role | Name | Student ID | GitHub |
|---|---|---|---|
| A | Fengyi Zhan | | |
| B | Kaile Shao | | |
| C | jianqiao Fang | | |
| D | xin Li | | |

Main text: 1324 conservative tokens / 1256 whitespace words; limit 1,500. Includes tables, diagram labels, citations and contribution statements.

Assessment draft: student IDs and GitHub accounts are intentionally blank. Individual tasks are allocated; completed contribution records and substantive review details remain to be confirmed.

Assessed application commit: 03fb9c5e41f80026c0ec66c28a92b09ed6f1251e

https://github.com/alwaysgoodz/aut-bug-report-helper

## 1. Project definition and scope

Incomplete bug reports create clarification work before debugging. Bettenburg et al. (2008) found mismatches between developers' information needs and reporters' submissions. Our helper prompts student teams for reproducible descriptions; reduced clarification is the intended value, to be evaluated rather than assumed. Topic approval was confirmed by the repository owner; no approval record is attached.

Reporters need understandable prompts, developers need reproduction details, testers need expected/actual outcomes, team leads need status visibility, and the lecturer needs traceable assessment evidence. Scope is seven fields (title, description, steps, expected behaviour, actual behaviour, environment, severity), local persistence, listing/status changes, completeness guidance and filtering. Authentication, public hosting, AI diagnosis, attachments and external tracker integration are excluded.

Four members, two short iterations and limited availability constrain delivery. Three engineering issues drive our choices: evolving guidance requirements, conflicting service/schema changes, and misleading or incomplete reports. The product is a single-instance local prototype; the shared repository coordinates development. No existing application code was imported; Python/SQLite and acknowledged research are reused.

## 2. Comparison and justified method selection

We compare plan-driven development with iterative-incremental development using agile practices against the same criteria (Table 1). Iteration describes repeated development and delivery; agile adds feedback and adaptation, so the labels are not mutually exclusive (Larman & Basili, 2003).

**Table 1. Alternatives under the project constraints**

| Decision factor | Plan-driven approach | Iterative-incremental with agile practices |
|---|---|---|
| Requirement change | Baseline fields and approve changes: predictable scope, but early guidance assumptions may cause rework. | Demonstrate reporting before guidance; reprioritise feedback between increments. Changes still consume fixed capacity. |
| Coordination | Documented responsibilities and interfaces support handovers; sequential work can delay integration feedback. | Shared backlog, short branches and frequent integration expose conflicts early; require reviewer availability. |
| Quality assurance | Up-front verification and traceability suit stable, consequential requirements; late user validation can miss usability problems. | Acceptance checks and reviews accompany each increment; repeated regression costs time and does not guarantee useful feedback. |
| Delivery effort | Detailed baselines are useful for storage contracts; comprehensive documentation competes with the small implementation budget. | Small working slices permit early learning; full framework ceremonies would be disproportionate for four people. |

We select two timeboxed increments, retaining a field contract and testable acceptance baseline while adapting guidance after demonstration. Boehm and Turner's (2003) risk framework supports this balance: small size, low criticality and uncertain usability favour feedback, while shared persistence needs deliberate design. Scope changes enter the backlog; only urgent defects interrupt an increment. We use prioritisation, reviews and retrospectives without claiming full Scrum adoption.

Research supports a conditional choice. Dybå and Dingsøyr's (2008) systematic review found limited empirical evidence, dominated by early XP studies; it cannot establish universal agile superiority. McIntosh et al. (2014) associate review participation with quality in large open-source systems, supporting substantive review rather than approval counts; causality and transfer to four students remain uncertain. Elazhary et al.'s (2022) three-company study shows CI implementation varies and can constrain feedback. Consequently, our pipeline checks acceptance and compilation on two Python versions; human review examines intent and omissions.

A distributed team would retain increments but add asynchronous decision records, explicit handovers and agreed review windows. Camara et al.'s (2024) distributed large-scale review supports tailoring, although its scale limits direct transfer. A regulated system would require stronger hazard analysis, requirement-test traceability and documented approvals; feedback cycles could remain, with more plan-driven assurance (Boehm & Turner, 2003).

## 3. Prioritised plan, design and risks

A/B/C/D map to the named members on the title page: requirements, interaction, service/storage and testing/CI respectively. Table 2 gives proposed author/reviewer pairs and person-hour estimates; BR-01 is the A1 trial reused in A2. Hours are team effort; owners do not work alone.

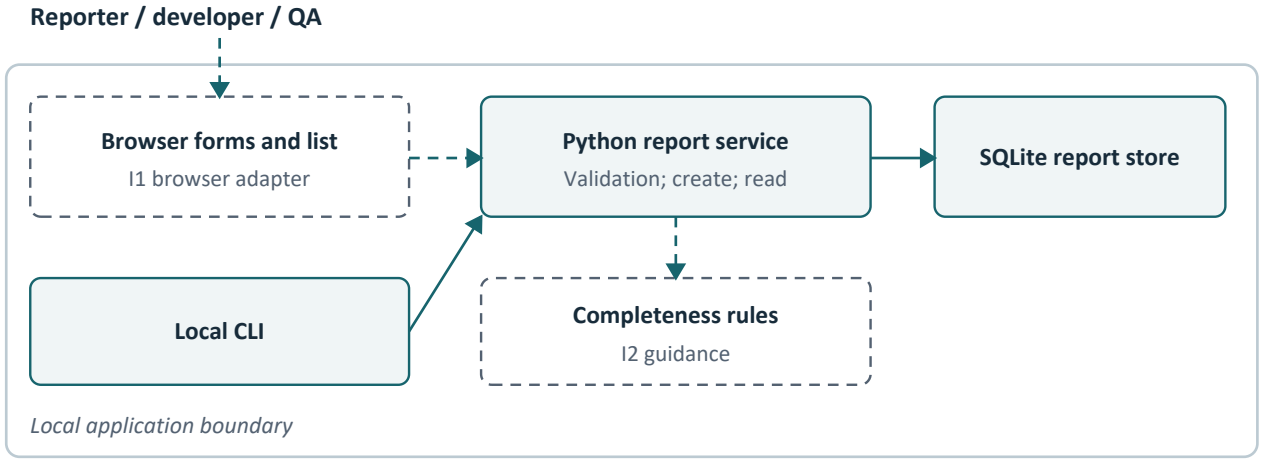
**Table 2. Backlog and testable acceptance criteria (E1)**

| Item / priority | Acceptance criteria | Iteration; hours; author/reviewer |
|---|---|---|
| BR-01 / Must | Persist seven fields, unique ID, UTC timestamp and Open status. Fresh-process read matches. Blank title/invalid severity inserts nothing. | I1; 4; C/D |
| BR-02 / Must | Browser create/list survives reload; details show all fields. Invalid input retains data with accessible errors; keyboard operation works; HTML text is escaped. | I1; 6; B/C |
| BR-03 / Must | Open/In progress/Resolved updates and reopening survive restart. Invalid status or unknown ID causes no mutation. | I1; 4; C/B |
| BR-04 / Must | Score round(100n/7) for six nonblank text fields plus valid severity: 6/7=86; 7/7=100. Identify missing fields; optional blanks remain savable. | I2; 5; B/D |
| BR-05 / Should | Severity/status filters combine with AND; Clear restores all; empty results are explicit; records remain unchanged. | I2; 3; C/B |
| BR-06 / Must | Clean checkout passes applicable checks. Record demonstration feedback, reproduction outcomes and exact revision evidence. | I1 2h; I2 4h; D+A/C |

Proposed capacity is 20 person-hours per two-day iteration: five hours each, subject to confirmation. I1 delivers create/list/status (16 hours plus four contingency); I2 delivers evaluated guidance/filtering (12 plus eight contingency). A1 setup/trial/report occur in Week 1; implementation continues in A2. Each increment ends with demonstration and one recorded retrospective action. Dependencies: BR-01 -> BR-02 -> BR-04; BR-03 precedes BR-05; BR-06 spans both.

Figure 1 shows the A1 reference and intended extensions. Shared service logic prevents CLI/browser validation drift. SQLite transactions and parameterised queries suit local persistence; explicit connection closure supports Windows cleanup. A pure scoring function is independently testable. Completeness measures presence, not correctness.

**Figure 1. A1 reference: solid = implemented; dashed = planned A2 extension**




If remaining Must effort exceeds capacity, A freezes additions and drops BR-05. Concurrent schema edits trigger paired integration led by C. A review waiting one working day triggers D to assign a backup non-author. A misleading high score triggers B to revise guidance; C checks restart persistence. These responses protect core functionality and review time.

Success requires all applicable acceptance checks passing and no critical unresolved defects. I2 plans four consenting peers using matched synthetic tasks with/without guidance in alternating order. Record time, field completeness and independent reproduction without clarification. Report descriptive outcomes and convenience-sample limitations; this evaluation has not run.

## 4. Workflow trial and individual contributions

Issue #1 preceded branch feat/1-create-report and PR #7 (E2). Assessed code reference 03fb9c5e41f80026c0ec66c28a92b09ed6f1251e passed 16 tests on both CI Python versions and compilation (E3). A fresh Windows snapshot recheck passed the same tests (E4). Checks cover separate-process persistence, rejected inputs without inserts, metadata and Unicode/SQL-like text.

The trial exposed Windows file-lock errors: a SQLite transaction context did not close test connections. Explicit closure resolved cleanup failures. The resulting review checklist checks resource lifetime as well as data correctness. This execution-driven correction illustrates feedback within a small increment; it was not a peer-discovered defect.

The workflow is Issue -> short branch -> tested PR -> non-author review -> response -> approved merge. Work in progress is limited to two implementations. Done requires acceptance evidence, latest-SHA approval and updated links. On 9 November, repository evidence was publicly accessible. PR #7 has fadeout12 approval on the assessed SHA, with an empty body and no comments; it remains draft/unmerged. Review rationale, reviewer tests and member attribution remain unverified (E5). Main is unprotected; review is a manual rule.

Codex assisted research, code, tests and drafting. Table 3 allocates individual work; members must execute it and link their own decisions/results (E6).

**Table 3. Individual contribution status**

| Member | Allocated work (25% planned each) | Evidence to record (pending) |
|---|---|---|
| Fengyi Zhan (A) | Scope/methods/backlog | Decision + activity link (E1). |
| Kaile Shao (B) | Interaction/input review | Findings + check log (E2). |
| jianqiao Fang (C) | Persistence/design review | Design/test record (E2). |
| xin Li (D) | Tests/CI/release evidence | Independent run (E3/E4). |

## References

Bettenburg, N., Just, S., Schröter, A., Weiss, C., Premraj, R., & Zimmermann, T. (2008). What makes a good bug report? In *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering* (pp. 308–318). Association for Computing Machinery. https://doi.org/10.1145/1453101.1453146

Boehm, B., & Turner, R. (2003). Using risk to balance agile and plan-driven methods. *Computer, 36*(6), 57–66. https://doi.org/10.1109/MC.2003.1204376

Camara, R., Marinho, M., & Moura, H. (2024). Agile tailoring in distributed large-scale environments using agile frameworks: A systematic literature review. *CLEI Electronic Journal, 27*(1), Article 8. https://doi.org/10.19153/cleiej.27.1.8

Dybå, T., & Dingsøyr, T. (2008). Empirical studies of agile software development: A systematic review. *Information and Software Technology, 50*(9–10), 833–859. https://doi.org/10.1016/j.infsof.2008.01.006

Elazhary, O., Werner, C., Li, Z. S., Lowlind, D., Ernst, N. A., & Storey, M.-A. (2022). Uncovering the benefits and challenges of continuous integration practices. *IEEE Transactions on Software Engineering, 48*(7), 2570–2583. https://doi.org/10.1109/TSE.2021.3064953

Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer, 36*(6), 47–56. https://doi.org/10.1109/MC.2003.1204375

McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2014). The impact of code review coverage and code review participation on software quality: A case study of the Qt, VTK, and ITK projects. In *Proceedings of the 11th Working Conference on Mining Software Repositories* (pp. 192–201). Association for Computing Machinery. https://doi.org/10.1145/2597073.2597076

## Appendix A. Supporting evidence index

Supporting links only; assessed explanations and limitations are in the main text.

| ID | Evidence link | Verified meaning / limit |
|---|---|---|
| E1 | Prioritised backlog | Six requirements, acceptance criteria, responsibilities, capacity and risk responses; original tracked Issues #1-6. |
| E2 | Tracked requirement: Issue #1 | Created before implementation; service/CLI scope and rejection/persistence criteria. |
| E2 | Reviewed change: PR #7 | Open draft; head 03fb9c5e41f80026c0ec66c28a92b09ed6f1251e. Non-author approval exists; substantive review evidence is absent. |
| E3 | Assessed code reference | 03fb9c5e41f80026c0ec66c28a92b09ed6f1251e; inspect this A1 snapshot rather than later A2 features. |
| E3 | CI run 41675675123 | Success on the assessed SHA: Python 3.11 and 3.12; 16 acceptance tests/job plus compilation. |
| E4 | Fresh snapshot check and verification | 9 November recheck, command exit codes, environment, public URL access and live review metadata. Local tests log is adjacent. |
| E5 | Non-author approval record | fadeout12 approved the assessed SHA on 6 November 2026. Empty body; no inline/discussion comments. Member mapping unconfirmed. |
| E6 | Individual activity ledger | Individual work allocated at 25% planned each; execution results and genuine activity links still need recording. |
| E7 | A1 setup and test instructions | Exact SHA checkout; no third-party application dependencies. Report regeneration and evidence verification instructions. |

Access and submission identity. The repository and sampled Issue/PR/CI/README pages returned HTTP 200 without authentication on 9 November 2026. Visibility may change; confirm access at submission. The assessed application reference is fixed above; later A2/UI branches are outside this A1 trial. The report is not submission-ready until verified individual contribution statements are completed and the team finalises the required title-page details. Public read access does not establish team write access.

Preparation assistance. Codex assisted source discovery/appraisal, report drafting, diagram preparation, implementation, testing and repository setup. Existing application code was not imported. No four-student authorship, substantive peer discussion or user-study results are inferred from AI-assisted activity. Team members must verify the material and apply the course rules for acknowledging assistance.